



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe Avance 3 - Problema 2

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

20 de abril 2026

1. Introducción

En este trabajo se implementó en GNU Octave el método de eliminación gaussiana para resolver un sistema lineal de la forma

$$Ax = b,$$

donde $A \in \mathbb{R}^{1000 \times 1000}$ es una matriz tridiagonal con diagonal principal igual a 5, y con subdiagonal y superdiagonal iguales a 1. Además, el vector $b \in \mathbb{R}^{1000}$ está dado por

$$b(i) = \frac{i}{10}, \quad i = 1, 2, \dots, 1000.$$

El objetivo principal fue obtener la solución aproximada del sistema y, en lugar de imprimir el vector solución, calcular e imprimir únicamente el error

$$\|Ax - b\|_2.$$

2. Estrategia utilizada en cada apartado

2.1. Construcción de la matriz A

La primera parte del ejercicio consistió en construir la matriz de coeficientes A . Como el enunciado indica que la matriz es tridiagonal, se utilizó una estrategia basada en vectores auxiliares:

- un vector para la diagonal principal con todos sus elementos iguales a 5,
- un vector para la superdiagonal con todos sus elementos iguales a 1,
- un vector para la subdiagonal con todos sus elementos iguales a 1.

Luego, la matriz se construyó usando la función `diag()`, que permite colocar un vector en una diagonal específica. Cabe mencionar que se usó una estrategia en la cual a partir de un vector de 100 valores se crearon la matriz principal y las secundarias.

2.2. Construcción del vector b

Para construir el vector b , se inicializó un vector columna de tamaño 1000 y luego se llenó usando un ciclo `for`, asignando en cada posición:

$$b(i) = \frac{i}{10}.$$

Esta parte se implementó de manera directa, siguiendo exactamente la definición dada en el enunciado.

2.3. Eliminación gaussiana

La estrategia aplicada en esta etapa fue trabajar con la matriz aumentada

$$[A \mid b],$$

para transformar el sistema original en un sistema triangular superior equivalente:

$$Ux = \tilde{b}.$$

En cada columna k , se realizaron operaciones elementales por filas para anular los elementos que se encuentran debajo del pivote. El multiplicador utilizado en cada paso fue:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}.$$

Después, se actualizó cada fila mediante:

$$F_i \leftarrow F_i - m_{ik}F_k.$$

Para mejorar la estabilidad del algoritmo y evitar divisiones entre cero, se incorporó pivoteo parcial. La estrategia fue, en cada paso k , buscar en la columna k el elemento de mayor valor absoluto desde la fila k hasta la fila final. Si ese elemento no estaba en la fila actual, se intercambiaban las filas. Pero luego se analizó que no es necesario hacer esto porque la matriz inicial siempre es diagonal. A pesar de eso el código se dejó comentado.

2.4. Sustitución hacia atrás

Una vez obtenida la matriz triangular superior U , se resolvió el sistema

$$Ux = \tilde{b}$$

mediante sustitución hacia atrás.

La estrategia fue calcular primero la última componente:

$$x_n = \frac{\tilde{b}_n}{u_{nn}},$$

y luego avanzar desde la última fila hacia la primera usando:

$$x_i = \frac{\tilde{b}_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}.$$

2.5. Cálculo del error

Finalmente, en lugar de imprimir el vector solución, se calculó el error pedido en el enunciado:

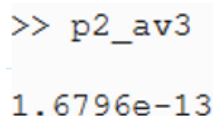
$$\text{error} = \|Ax - b\|_2.$$

La estrategia consistió en usar la matriz original A , el vector original b y la solución calculada x . Luego se aplicó la norma euclidiana con la función `norm(...,2)` de Octave.

3. Comandos, funciones y paquetes utilizados

3.1. Bugs resueltos

Como la tarea pedía implementar el algoritmo visto en clase se realizó inicialmente una implementación con un for principal y 2 for anidados. Pero al intentar resolver esto con una matriz de 1000x1000 el programa no lograba finalizar. Se dió un tiempo de más de 30 minutos y el programa nunca dió un resultado de error. Esto se solucionó por medio de vectorización. Se vectorizó el proceso que convertía en cero los coeficientes de la columna. Con esto solo se tenía un for anidado dentro del principal y la función lograba dar un resultado final.



```
>> p2_av3
1.6796e-13
```

Figura 1: Resultado del error una vez que se usó la vectorización.

3.2. Funciones de GNU Octave utilizadas

A continuación, se describen las funciones más importantes utilizadas durante la implementación:

- `zeros(n,m)`: se utilizó para inicializar matrices y vectores llenos de ceros.
- `ones(n,1)`: se utilizó para crear vectores con todos sus elementos iguales a 1.
- `diag(v)`: permitió construir la diagonal principal de la matriz.
- `diag(v,1)` y `diag(v,-1)`: se usaron para crear la superdiagonal y la subdiagonal.
- `rows(A)`: permitió obtener el número de filas de la matriz.
- `abs(...)`: se usó al momento de buscar el pivote de mayor magnitud.
- `max(...)`: permitió encontrar el máximo valor absoluto en la columna relevante.
- `norm(A*x-b,2)`: se utilizó para calcular el error euclidiano.

3.3. Estructuras de control utilizadas

- `for`: se utilizó en la construcción del vector b , en la eliminación gaussiana y en la sustitución hacia atrás.
- `if`: se utilizó para verificar si el pivote era válido y para decidir cuándo intercambiar filas.

3.4. Paquetes de LaTeX utilizados

En este documento se utilizaron los siguientes paquetes:

- `amsmath`, `amssymb`: para escribir expresiones matemáticas.
- `geometry`: para ajustar los márgenes del documento.
- `listings`: para mostrar fragmentos de código de Octave.
- `xcolor`: para dar formato visual al código.
- `hyperref`: para enlaces y mejor navegación en el PDF.
- `setspace`: para mejorar el espaciado entre líneas.
- `babel`: para adaptar el documento al idioma español.

4. Fragmento representativo del código en Octave

A continuación se muestra un fragmento representativo del desarrollo realizado:

```
function p2_av3
    coefs = matDiagonal();
    results = resVector();

    A_original = coefs;
    b_original = results;

    matAumentada = [coefs, results];
    m = rows(coefs);

    [U, b] = eliminacionGaussina(matAumentada, m);
    x = sustAtras(U, b, m);

    error_final = norm(A_original*x - b_original, 2);
    disp(error_final);
endfunction
```

5. Conclusión

Se implementó el método de eliminación gaussiana en GNU Octave para resolver un sistema lineal de gran tamaño. El procedimiento incluyó la construcción de la matriz tridiagonal, la generación del vector b , la triangularización mediante eliminación gaussiana, la sustitución hacia atrás y el cálculo del error final. El pivoteo parcial no fue de utilidad en este caso y se tuvo que usar vectorización para que el programa pudiera concretar el resultado.

El resultado solicitado por el enunciado no fue el vector solución, sino el valor de

$$\|Ax - b\|_2,$$

el cual permite verificar la calidad de la solución obtenida.